

PSYC 51.17: Models of Language and Communication

# Lecture 24: Retrieval Augmented Generation

Grounding LLMs in External Knowledge

PSYC 51.17: Models of Language and Communication

Week 9

Week 9

# Today's Journey

What we'll cover

1. **The Problem:** Why parametric memory isn't enough
2. **RAG Basics:** Retrieve, Augment, Generate
3. **Implementation:** Building RAG systems step-by-step
4. **Advanced Techniques:** Self-RAG, Corrective RAG, HyDE
5. **Production Challenges:** Making RAG work in the real world

Week 9

- **Knowledge cutoff:** No information after training date
- **Hallucinations:** Models confidently generate false information
- **No source attribution:** Can't cite where information comes from
- **Expensive updates:** Retraining for new information costs millions
- **Privacy concerns:** Sensitive data baked into parameters
- **Domain specificity:** Limited knowledge of specialized domains

## Example: Knowledge Cutoff Problem

User Query (Dec 2024)

"Who won the 2024 US Presidential election?"

### Parametric-only LLM:

{ "I apologize, but my knowledge was last updated in April 2023, so I cannot tell you about the 2024 election results."

Or worse: Hallucinates an answer!

### RAG-enhanced LLM:

{ "According to CNN (retrieved Nov 6, 2024), [actual winner] won the 2024 US Presidential election with [details]."

Provides: Fresh info + source!

Key Insight

Retrieval provides a **dynamic, updatable** knowledge base without retraining!

# Retrieval Augmented Generation: Definition

RAG (Lewis et al., 2020)

A technique that enhances LLMs by retrieving relevant documents from an external knowledge base and using them to inform generation.

**Core Idea:** Instead of relying only on learned parameters, the model can "look things up"!

**Traditional LLM:**

- Question → Model → Answer

**RAG Pipeline:**

- Question → Retrieve Docs
- Docs + Question → Model

# RAG Architecture



**Worked Example: "What causes the Northern Lights?"**

| Step     | Action                  | Result                                                             |
|----------|-------------------------|--------------------------------------------------------------------|
| 1. Embed | Convert query to vector | <div>[0.12,<br/>-0.45, 0.78,<br/>... ]</div> <div>(384 dims)</div> |



# RAG: Step-by-Step Walkthrough

**Query:** "What is the capital of Kazakhstan?"

```
1 # Step 1: Embed the query
2 query = "What is the capital of Kazakhstan?"
3 query_embedding = embedding_model.encode(query)
4 # Result: numpy array of shape (384,)
5
6 # Step 2: Search vector database
7 results = vector_db.search(query_embedding, top_k=3)
8 # Returns: [
9 # {"text": "Astana is the capital of Kazakhstan ... ", "score": 0.94},
10 # {"text": "Kazakhstan's capital moved from Almaty ... ", "score": 0.89},
11 # {"text": "The city was renamed Nur-Sultan in 2019 ... ", "score": 0.85}
12 # ]
13
14 # Step 3: Build augmented prompt
15 context = "\n".join([r["text"] for r in results])
16 prompt = f""Answer based on the context below.
```

7

Week 9

continued...

# RAG: Step-by-Step Walkthrough

```
17 Context: {context}
18 Question: {query}
19 Answer: ""
20
21 # Step 4: Generate with LLM
22 response = llm.generate(prompt)
23 # "Astana (previously known as Nur-Sultan) is the capital of Kazakhstan."
```

Week 9



# RAG Components Deep Dive

## 1. Document Processing

```
1 # Chunking example
2 from langchain.text_splitter
  import
  RecursiveCharacterTextSplitter
3
4 splitter =
  RecursiveCharacterTextSplitter(
5     chunk_size=500, # Target size
6     chunk_overlap=50, # Overlap
    between chunks
7     separators=["\n\n", "\n", ". "
8 ]
```

## 2. Embedding & Storage

```
1 from sentence_transformers
  import SentenceTransformer
2 import chromadb
3
4 # Create embeddings
5 model =
  SentenceTransformer('all-MiniLM
  L6-v2')
6 embeddings =
  model.encode(chunks)
7
8 # Store in vector database
```

# RAG Implementation Example

## Basic RAG with LangChain:

```
1 from langchain.vectorstores import Chroma
2 from langchain.embeddings import HuggingFaceEmbeddings
3 from langchain.llms import HuggingFacePipeline
4 from langchain.chains import RetrievalQA
5 from langchain.document_loaders import TextLoader
6 from langchain.text_splitter import RecursiveCharacterTextSplitter
7
8 # 1. Load and chunk documents
9 loader = TextLoader('knowledge_base.txt')
10 documents = loader.load()
11
12 text_splitter = RecursiveCharacterTextSplitter(
13     chunk_size=512,
14     chunk_overlap=50
15 )
```

Week 9

10

continued...

# RAG Implementation Example

```
16 chunks = text_splitter.split_documents(documents)
17
18 # 2. Create embeddings and vector store
19 embeddings = HuggingFaceEmbeddings(
20     model_name="sentence-transformers/all-MiniLM-L6-v2"
21 )
22 vectordb = Chroma.from_documents(
23     documents=chunks,
24     embedding=embeddings,
25     persist_directory="./chroma_db"
26 )
27
28 # 3. Set up retriever
29 retriever = vectordb.as_retriever(
30     search_type="similarity",
```

# RAG Implementation Example

```
31 | search_kwargs={"k": 3} # Retrieve top 3 chunks  
32 | )
```

Week 9

# RAG Implementation (cont.)

```
1 # 4. Create LLM
2 llm = HuggingFacePipeline.from_model_id(
3     model_id="meta-llama/Llama-2-7b-chat-hf",
4     task="text-generation",
5     model_kwargs={"temperature": 0.7, "max_length": 512}
6 )
7
8 # 5. Create RAG chain
9 qa_chain = RetrievalQA.from_chain_type(
10     llm=llm,
11     retriever=retriever,
12     return_source_documents=True,
13     chain_type="stuff" # How to combine documents
14 )
15
16 # 6. Query the system
17 query = "What is retrieval augmented generation?"
18 result = qa_chain({"query": query})
```

13

Week 9

continued...

## RAG Implementation (cont.)

```
19  
20 print("Answer:", result['result'])  
21 print("\nSources:")  
22 for doc in result['source_documents']:  
23     print(f"- {doc.metadata['source']}: {doc.page_content[:100]} ... ")
```

*Tutorial: HuggingFace Advanced RAG -  
[https://huggingface.co/learn/cookbook/advanced\\_rag](https://huggingface.co/learn/cookbook/advanced_rag)*

Week 9

14

...continued



# Evolution of RAG Approaches



| Approach       | Key Innovation       | When to Retrieve     |
|----------------|----------------------|----------------------|
| Naive RAG      | Always retrieve      | Every query          |
| Self-RAG       | Model decides        | Only when needed     |
| Corrective RAG | Verify relevance     | Always, but filter   |
| Agentic RAG    | Multi-step reasoning | Tool-based decisions |

Trend

Moving from always-retrieve to **adaptive, self-correcting** retrieval systems!

# Self-RAG: Adaptive Retrieval

**Key Innovation:** Model decides when to retrieve

## Special Tokens Learned:

- **[Retrieve]** - Need external info?
- **[Relevant]** - Is retrieved doc useful?
- **[Support]** - Does doc support answer?
- **[Useful]** - Is answer helpful?

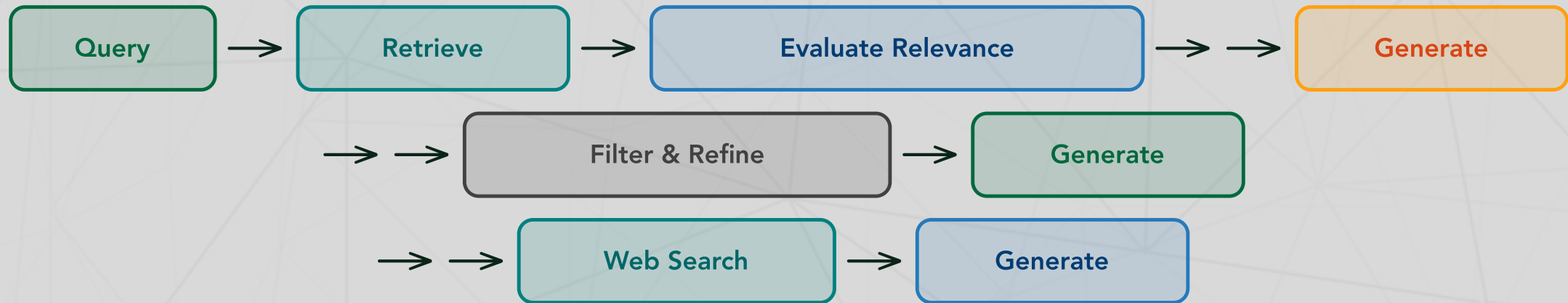
## Worked Example:

|   |                                             |
|---|---------------------------------------------|
| 1 | Q: What's 2+2?                              |
| 2 | [Retrieve: No] # No retrieval needed        |
| 3 | A: 4                                        |
| 4 |                                             |
| 5 | Q: Who won the 2024 Olympics?               |
| 6 | [Retrieve: Yes] # Need current info         |
| 7 | [Retrieved: "Paris 2024 Olympic Games ..."] |
| 8 | [Relevant: Yes] # Doc is on topic           |

16

# Corrective RAG (CRAG)

**Problem:** Sometimes retrieved documents are irrelevant or misleading!



**Worked Example:**

1 | # Query: "Latest COVID vaccine recommendations"

Key Idea

Don't blindly trust retrieval! Verify relevance and have fallback strategies.

# Comparing RAG Approaches

| Approach       | When Retrieve   | Filtering         | Latency | Best For          |
|----------------|-----------------|-------------------|---------|-------------------|
| Naive RAG      | Always          | None              | Low     | Simple Q&A        |
| Self-RAG       | Model decides   | Self-reflection   | Medium  | Adaptive needs    |
| Corrective RAG | Always + verify | Relevance scoring | High    | High precision    |
| HyDE           | Via hypothesis  | Similarity        | Medium  | Complex queries   |
| Agentic RAG    | Tool-based      | Multi-step        | Highest | Complex workflows |

## Trade-offs Example:

1 | Simple FAQ bot → Naive RAG (fast, cheap)

Rule of Thumb

Start with Naive RAG. Add complexity only when you

# Chunking Strategies

How you split documents dramatically affects retrieval quality!

## Fixed-size (Simple)

```
1 # Split every 500 chars
2 chunks = [text[i:i+500]
3   for i in range(0,
4     len(text), 500)]
5 # Problem: "The
  mitochondria is the power-
  "
# "house of the cell." ←
split mid-sentence!
```

## Recursive (Better)

## Semantic (Best Quality)

```
1 # Find natural breakpoints
  using embeddings
2 from
  langchain.text_splitter
  import SemanticChunker
3
4 chunker =
  SemanticChunker(embeddings)19
5 # Groups sentences with
  similar meaning
```

# Embedding Models for Retrieval

## Choosing the Right Embedding Model:

| Model            | Dims | Size  | Speed  | Quality   |
|------------------|------|-------|--------|-----------|
| all-MiniLM-L6-v2 | 384  | 90MB  | Fast   | Good      |
| BGE-large-en     | 1024 | 1.3GB | Medium | Excellent |



# Embedding Models for Retrieval

```
10 scores = bm25.get_scores(query.split())
11 # Finds: "Headaches can be caused by ..." (exact keyword match)
12
13 # Hybrid: Combine both for best results!
14 final_score = 0.7 * dense_score + 0.3 * sparse_score
```

Week 9

# Vector Databases

**Purpose:** Fast similarity search over millions of embeddings

## Quick Start with ChromaDB:

```
1 import chromadb
2
3 # Create client and collection
4 client = chromadb.Client()
5 collection =
6 client.create_collection("my_docs")
7
8 # Add documents (auto-embeds!)
9 collection.add(
10     documents=["Paris is in France",
11               "Berlin is in Germany"],
12     ids=["doc1", "doc2"]
13 )
```

## Choosing a Vector DB:

| Use Case       | Best Choice      |
|----------------|------------------|
| Prototyping    | ChromaDB         |
| Production     | Pinecone, Qdrant |
| Self-hosted    | FAISS, Milvus    |
| Graph + Vector | Weaviate         |

## Key Features:

- Query latency (<50ms for 1M docs)
- Metadata filtering
- Persistence & backups

# Prompt Engineering for RAG

## Template for Grounded Generation:

```
1 RAG_PROMPT = ""You are a helpful assistant. Answer the question based ONLY on
2 the context provided below. If the answer is not in the context, say
3 "I don't have that information."
4
5 Context:
6 {context}
7
8 Question: {question}
9
10 Instructions:
11 - Use only information from the context above
12 - Cite sources using [1], [2], etc.
13 - Be concise and accurate
```

# Prompt Engineering for RAG

```
14
15 Answer: ""
16
17 # Example usage
18 context = ""[1] The Eiffel Tower was completed in 1889 for the World's Fair.
19 [2] It stands 330 meters tall and was the world's tallest structure until 1930.
20 [3] Gustave Eiffel's company designed and built the tower.""
21
22 question = "When was the Eiffel Tower built?"
23
24 response = llm.generate(RAG_PROMPT.format(context=context, question=question))
25 # "The Eiffel Tower was completed in 1889 for the World's Fair [1]."
```

## Key Elements

1. Explicit grounding instruction, 2. Source citation format, 3. Fallback for missing info

# Production Challenges

## Performance Challenges:

- **Cost:** Embedding generation + storage + inference
- **Latency:** Retrieval adds 50-200ms
- **Context limits:** LLM window size

## Solutions:

- Cache embeddings
- Semantic caching (similar queries)
- Incremental indexing
- Re-ranking pipelines
- Efficient embedding models<sup>25</sup>

doc?

- **NDCG (Normalized Discounted Cumulative Gain):**  
Ranked quality

## Generation Quality:

- **Factual accuracy:** Are answers correct?
- **Faithfulness:** Does answer match retrieved docs?
- **Relevance:** Does answer address the question?
- **Citation quality:** Are sources correctly attributed?

26

## End-to-End:



- Too much irrelevant context
- Context too long for LLM
- Important info not in retrieved chunks

### **3. Generation Issues**

- Ignores retrieved context
- Hallucinates despite good context
- Incorrect citations

## \item **Code Retrieval**

- Embed code snippets
- Retrieve relevant functions/examples
- Code completion and debugging

## \item **Structured Data**

- Tables, databases
- Knowledge graphs
- SQL generation from natural language

# Graph RAG: Documents + relationships

## Advantages:

- Capture entity relationships
- Multi-hop reasoning ( $A \rightarrow B \rightarrow C$ )
- Better for complex queries
- Explicit knowledge structure

## Implementation:

1. Extract entities and relations from documents
2. Build knowledge graph
3. Traverse graph during retrieval

# HyDE: Hypothetical Document Embeddings

**Clever trick: Generate a hypothetical answer first, then retrieve!**

1 | # Standard RAG: Query → Retrieve → Generate

Why It Works

**Question:** "aurora borealis causes" (query-like)

**Hypothesis:** "charged particles from sun interact with atmosphere" (document-like)

Answers are more similar to documents than questions are!

*Reference: Gao et al. (2022) - "Precise Zero-Shot Dense Retrieval without Relevance Labels"*

# RAG vs Fine-Tuning

When should you use RAG vs fine-tuning your model?

## Use RAG when:

- Knowledge changes frequently
- Need citations/provenance
- Privacy concerns (data in DB, not weights)
- Large knowledge base
- Multi-domain applications
- Want to update without retraining

## Use Fine-Tuning when:

- Need specific style/behavior
- Low latency critical
- Small, stable knowledge domain
- Specialized reasoning
- Domain-specific language
- Want fully self-contained model

## 2. Long-context RAG

- Models with 1M+ token windows
- Entire books as context
- Retrieval still useful for efficiency

## 3. Personalized RAG

- User-specific knowledge bases
- Privacy-preserving retrieval



### 3. Many variants exist

- Naive RAG → Self-RAG → Corrective RAG → Agentic RAG

### 4. Key components matter

- Chunking strategy, embedding model, vector DB

1. **Lewis et al. (2020):** Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks [[arXiv](#)]
2. **Asai et al. (2023):** Self-RAG: Learning to Retrieve, Generate, and Critique [[arXiv](#)]

### **Recommended:**

- Yan et al. (2024): Corrective RAG [[arXiv](#)]
- Gao et al. (2022): Precise Zero-Shot Dense Retrieval (HyDE) [[arXiv](#)]
- HuggingFace RAG Tutorial [[Tutorial](#)]

# PSYC 51.17: Models of Language and Communication

Questions?

Next: Mixture of Experts!

Week 9